UNIVERSIDAD INDUSTRIAL DE SANTANDER
FACULTAD DE INGENIERÍAS FÍSICO-MECÁNICAS
ESCUELA INGENIERÍA DE SISTEMAS E INFORMÁTICA
INGENIERÍA EN CIENCIA DE DATOS: ESTRUCTURAS DE DATOS

Contexto

En el procesamiento de flujos continuos de datos (*data streams*) —tales como la telemetría de sensores, transacciones en tiempo real o registros de auditoría en servidores— es habitual analizar únicamente un bloque fijo correspondiente a las observaciones más recientes. A esta abstracción se le conoce como **ventana deslizante** (*sliding window*). Asimismo, en tareas de análisis suele requerirse la inspección de los eventos desde el registro más reciente hacia el más antiguo (orden inverso de llegada).

Descripción del Requerimiento

Escriba un programa en **C** que simule el procesamiento de un flujo de enteros positivos leídos desde la consola, el cual finaliza con el valor indicador `-1`. El programa debe mantener una **Cola** que represente una ventana deslizante de tamaño máximo variable K.

Por cada entero leído del flujo, el programa debe realizar las siguientes operaciones:

1. **Control de capacidad:** Si la Cola alcanza el tamaño máximo K, se debe extraer el elemento más antiguo (`dequeue`).
2. **Ingreso de dato:** Insertar el nuevo valor en la Cola (`enqueue`).
3. **Visualización invertida:** Invocando la función obligatoria:

```
1 void imprimirVentanaInvertida(cola c);
```

   Imprimir por pantalla los elementos de la ventana actual ordenados estrictamente desde el más reciente hasta el más antiguo.

Mecanismo Interno e Invariante de Estructura

Para lograr la impresión en orden inverso sin destruir ni alterar el estado original de la Cola, la función `imprimirVentanaInvertida` debe implementar el siguiente procedimiento algorítmico:

1. Desencolar uno a uno los elementos de la Cola trasladándolos a una **Pila auxiliar**.
2. Recorrer e imprimir los elementos aprovechando la propiedad LIFO (*Last-In, First-Out*) de la Pila.
3. Restaurar la Cola a su estado y orden original antes de que la función retorne la ejecución al hilo principal.

Ejemplo de Ejecución

Entrada de prueba:

```
10 20 30 40 50 -1
```

Traza de ejecución y salida esperada:

```
Ingresa 10 -> Ventana invertida: 10
Ingresa 20 -> Ventana invertida: 20 10
Ingresa 30 -> Ventana invertida: 30 20 10
Ingresa 40 -> Ventana invertida: 40 30 20 10
Ingresa 50 -> Ventana invertida: 50 40 30 20
```

Nota: Al ingresar el valor `50`, el elemento `10` es descartado automáticamente de la estructura por superar el límite $K = 4$.

Especificaciones Técnicas y Restricciones

- **Encapsulamiento y Nivel de Abstracción:** Tanto la función `main` como la función `imprimirVentanaInvertida` deben interactuar con las estructuras **únicamente mediante la interfaz pública** de sus TADs (`enqueue`, `dequeue`, `push`, `pop`, `isEmpty`, `crear`, `destruir`). Queda estrictamente prohibido manipular punteros internos de los nodos (`nodo->next`, `pila->top`, `cola->front`) por fuera de las funciones propias del TAD.
- **Gestión de Memoria:** Todas las estructuras instanciadas (la Cola principal de la ventana y las Pilas auxiliares creadas durante la inversión) deben liberar su memoria dinámicamente al finalizar su uso (`free`), garantizando cero fugas de memoria (*memory leaks*).
- **Casos de Borde:** Si el primer valor ingresado es `-1` (flujo vacío), el programa debe finalizar limpiamente sin realizar impresiones ni generar errores de ejecución o *segmentation faults*.
- **No está permitido el uso de arreglos:** Estáticos ni dinámicos